

Aula 07 — Um objeto coordenando vários outros

Como um objeto pode coordenar vários colaboradores sem assumir as responsabilidades deles?

Um item já sabe colaborar

```
Produto teclado = new Produto("Teclado", 150.0);  
ItemPedido itemTeclado = new ItemPedido(teclado, 2);  
  
Produto mouse = new Produto("Mouse", 80.0);  
ItemPedido itemMouse = new ItemPedido(mouse, 1);
```

Cada item sabe calcular o próprio subtotal.

ATIVIDADE

PONTO DE PARTIDA

E se o pedido tiver três itens?

```
double total = itemTeclado.calcularSubtotal()  
               + itemMouse.calcularSubtotal();
```

O que precisaria mudar quando um novo item entrasse no pedido?

O problema não está na soma

Main

cria objetos e conhece cada item

Pedido

ainda não existe no modelo

Falta um responsável pelo conjunto.

ATIVIDADE

RESPONSABILIDADE

Quem deveria conhecer os itens?

1. Produto , porque fornece o preço;
2. Main , porque cria os objetos;
3. Pedido , porque representa o conjunto.

Escolha uma alternativa e justifique com o que cada objeto representa.

A decisão

`Pedido` representa a unidade que reúne os itens.

Ele deve:

- manter os itens;
- receber novos itens;
- coordenar o cálculo do total.

Cada `ItemPedido` continua responsável pelo próprio subtotal.

Conceito-chave — objeto coordenador

Um objeto coordena colaboradores quando mantém o conjunto relevante e solicita a cada participante o comportamento necessário para cumprir uma responsabilidade do conjunto.

Coordenar não é copiar o estado nem refazer o trabalho dos colaboradores.

REPRESENTAR O CONJUNTO

Quantos campos cabem num pedido?

```
private ItemPedido primeiroItem;  
private ItemPedido segundoItem;  
private ItemPedido terceiroItem;
```

Essa estrutura fixa antecipadamente o tamanho do pedido.

Precisamos de uma coleção

Um único campo deve manter várias referências para objetos `ItemPedido` .

```
List<ItemPedido> itens;
```

`List` expressa a coleção em sequência; `ItemPedido` expressa o tipo dos elementos.

List e ArrayList

```
import java.util.ArrayList;
import java.util.List;

private List<ItemPedido> itens;

public Pedido() {
    itens = new ArrayList<>();
}
```

`List` é o tipo da coleção. `ArrayList` é a implementação concreta criada com `new`.

ATIVIDADE

REPRESENTAR O CONJUNTO

Uma lista vazia é um problema?

Um `Pedido` acabou de nascer e ainda não recebeu itens.

1. Qual total você espera?
2. Por que a lista vazia pode ser um estado coerente?

REPRESENTAR O CONJUNTO

O estado inicial pode ser coerente

- não há itens a percorrer;
- não há subtotal a acumular;
- o total é 0.0 ;
- o pedido já está pronto para receber colaboradores.

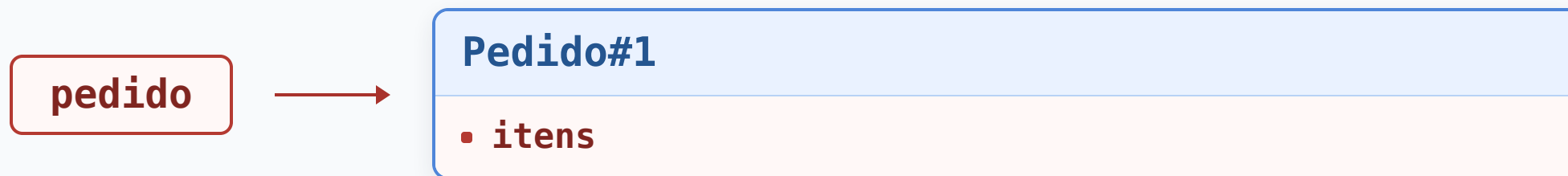
ADICIONAR COLABORADORES

O pedido recebe referências

```
pedido.adicionarItem(itemTeclado);  
pedido.adicionarItem(itemMouse);
```

Essas chamadas precisam criar novos itens ou guardar referências para itens que já existem?

O conjunto aponta para os objetos existentes



``add`` guarda a referência recebida. Não aparece ``new ItemPedido(...)``.

ATIVIDADE

ADICIONAR COLABORADORES

Acompanhe as referências

Depois de adicionar `itemTeclado` e `itemMouse` :

- quantos objetos `ItemPedido` foram criados pelas chamadas?
- o que a lista guarda?
- o que acontece se o mesmo item for adicionado novamente?

Uma coleção também contém referências

```
[0] → itemTeclado  
[1] → itemMouse
```

- as chamadas a `adicionarItem` não criam itens;
- a lista guarda referências existentes;
- uma segunda adição cria outra posição para a mesma identidade.

Como calcular sem listar item por item?

```
double total = 0.0;

for (ItemPedido item : itens) {
    total += item.calcularSubtotal();
}
```

O pedido conhece o conjunto. O item conhece o subtotal.

Leia o for aprimorado

```
for (ItemPedido item : itens) {  
    total += item.calcularSubtotal();  
}
```

Leia: “para cada `ItemPedido`, chamado temporariamente de `item`, presente em `itens`”.

A cada repetição, `item` recebe a referência para o próximo objeto.

O fluxo de responsabilidades



O pedido não precisa conhecer a multiplicação interna do subtotal.

ATIVIDADE

COORDENAR O PERCURSO

Preveja o percurso

Com dois itens no pedido:

1. quantas chamadas a `calcularSubtotal()` acontecem?
2. quantas chamadas a `getPreco()` acontecem?
3. quem conhece o conjunto de subtotais?
4. qual é o total?

Uma chamada por colaborador

- `calcularSubtotal()` é chamado duas vezes;
- `getPreco()` é chamado duas vezes;
- `Pedido` conhece o conjunto de subtotais;
- o total é $300.0 + 80.0 = 380.0$;
- um pedido vazio devolve `0.0` .

Duas soluções podem devolver 380.0

```
// solução colaborativa  
total += item.calcularSubtotal();
```

```
// solução que invade a regra do item  
total += item.getProduto().getPreco()  
        * item.getQuantidade();
```

Qual delas preserva melhor a distribuição de responsabilidades?

ARMADILHA

COORDENAR NÃO É FAZER

Armadilha — o coordenador refaz o subtotal

Pedido precisa saber quais colaboradores participarão e qual operação solicitar.
Ele não precisa aprender como preço e quantidade são combinados.

Coordenar $\neq$ absorver a responsabilidade dos colaboradores.

A mesma ideia em outro domínio

Uma **Turma** reúne vários **Aluno**. Cada aluno conhece suas notas e a regra de aprovação. A turma precisa contar quantos alunos estão aprovados.

1. Quem mantém a lista?
2. Qual operação a turma solicita?
3. Quem percorre a coleção?
4. Por que a turma não recalcula a média?

A responsabilidade continua distribuída

Pedido	Turma
percorre ItemPedido	percorre Aluno
solicita calcularSubtotal()	solicita estaAprovado()

- Turma mantém a lista e conta os resultados;
- Aluno conhece seu estado e a regra de aprovação;
- Turma não acessa notas para refazer a decisão.

Síntese

- `Pedido` mantém uma coleção de referências para seus itens;
- `ArrayList` é o objeto concreto que armazena a sequência;
- adicionar um item existente não cria cópia;
- `Pedido` percorre e coordena;
- `ItemPedido` calcula o subtotal;
- `Produto` fornece o preço;
- coordenar significa organizar chamadas sem absorver responsabilidades.

Próximo passo

No Laboratório 07, o Projeto 1 ganhará **Pedido**, uma coleção de itens e o cálculo coordenado do total.

Tente construir o responsável pelo conjunto antes de abrir as dicas.